

FinGuard — Guide d'installation & d'utilisation

**Système Intelligent de Gestion des
Transactions Bancaires et de
Détection de Fraude**

Ce guide explique, pas à pas, comment **installer l'application sur votre machine** (frontend, backend, base de données) puis comment **utiliser chaque espace** (client, employé, administrateur). L'interface est entièrement en **français**.

Sommaire

1. [Prérequis](#)
 2. [Installation sur votre machine](#)
 3. [Base de données](#)
 4. [Comptes de démonstration](#)
 5. [Comprendre le flux de fraude](#)
 6. [Utilisation — Espace CLIENT](#)
 7. [Utilisation — Espace EMPLOYÉ](#)
 8. [Utilisation — Espace ADMIN](#)
 9. [Scénarios à essayer \(démonstration\)](#)
 10. [Tests, documentation API et captures](#)
 11. [Dépannage](#)
-

1. Prérequis

Outil	Version	Rôle
Node.js	≥ 22 (recommandé)	Exécute le backend (le runtime de démonstration utilise node:sqlite, disponible à partir de Node 22) et le frontend
npm	≥ 9 (fourni avec Node)	Installe les dépendances
Git	toute version récente	Récupère le code
MySQL	Optionnel 8+	Uniquement pour un déploiement « production » via Prisma (voir §3)

Vérifiez vos versions :

```
node -v    # doit afficher v22.x ou plus
npm -v
```

Pourquoi Node 22 ? Le mode démonstration exécute la base via le module natif node:sqlite (aucun serveur de base de données à installer). Pour un déploiement MySQL/Prisma, Node 18+ suffit.

Ports utilisés : **4000** (API) et **3000** (site web).

2. Installation sur votre machine

2.1 Récupérer le code

```
git clone https://github.com/NGA828/FINGUARD.git
cd FINGUARD
```

2.2 Backend (API NestJS)

```
cd backend/banking-api
npm install          # installe les dépendances
npm run db:setup     # crée le schéma + insère les données de démonstration
npm run start        # démarre l'API
```

L'API démarre sur **http://localhost:4000/api** (message FinGuard API démarrée).

- **Documentation Swagger interactive** : <http://localhost:4000/api/docs>
- **OpenAPI (JSON)** : <http://localhost:4000/api/docs-json>

Scripts utiles (dans backend/banking-api) :

Commande	Effet
npm run db:init	(Re)crée uniquement le schéma de la base
npm run db:seed	Insère les données de démonstration
npm run db:reset	Schéma vierge + seed (reprenez par ici pour recommencer à zéro)
npm run test:e2e	10 tests de bout en bout contre l'API en fonctionnement

2.3 Frontend (site Next.js)

Ouvrez **un second terminal** :

```
cd frontend/banking-web
npm install
npm run dev          # mode développement → http://localhost:3000
```

Pour un lancement « production » :

```
npm run build
npm run start        # → http://localhost:3000
```

Le site **proxie automatiquement** toutes les requêtes `/api/*` vers l'API sur le port 4000 : aucun réglage CORS ou d'adresse à faire.

2.4 Vérification rapide

1. Ouvrez <http://localhost:3000> → la page d'accueil FinGuard s'affiche.
 2. Cliquez sur « **Accéder à mon espace** » (ou allez sur `/auth/login`).
 3. Connectez-vous avec `admin@finguard.com` / `Admin123!` → le tableau de bord administrateur apparaît. OK L'installation fonctionne.
-

3. Base de données

3.1 Mode démonstration (zéro configuration)

Par défaut, l'application utilise **SQLite** via `node:sqlite` : un simple fichier (backend/banking-api/data/), créé automatiquement par `npm run db:setup`.
Aucun serveur de base de données n'est nécessaire. Le DDL exécuté (src/database/schema.sql) est l'image 1:1 du schéma MySQL de production.

3.2 Mode production (MySQL + Prisma)

Le schéma canonique MySQL est fourni dans `backend/banking-api/prisma/schema.prisma`
(12 modèles : User, Customer, Beneficiary, Employee, Account, Transaction, FraudAnalysis, FraudAlert, Dispute, Notification, AuditLog, SystemConfig, FraudRule).

```
# 1. Créer la base MySQL (exemple)
mysql -u root -p -e "CREATE DATABASE finguard CHARACTER SET utf8mb4;"

# 2. Déclarer la connexion
cd backend/banking-api
echo 'DATABASE_URL="mysql://utilisateur:motdepasse@localhost:3306/finguard"' > .env

# 3. Pousser le schéma
npx prisma db push --schema prisma/schema.prisma
```

La logique métier repose sur une couche repository isolée (src/database/) : le branchement sur Prisma consiste à remplacer l'adaptateur SQLite par l'adaptateur Prisma, les entités et champs étant identiques.

3.3 Variables d'environnement du backend (`backend/banking-api/.env`)

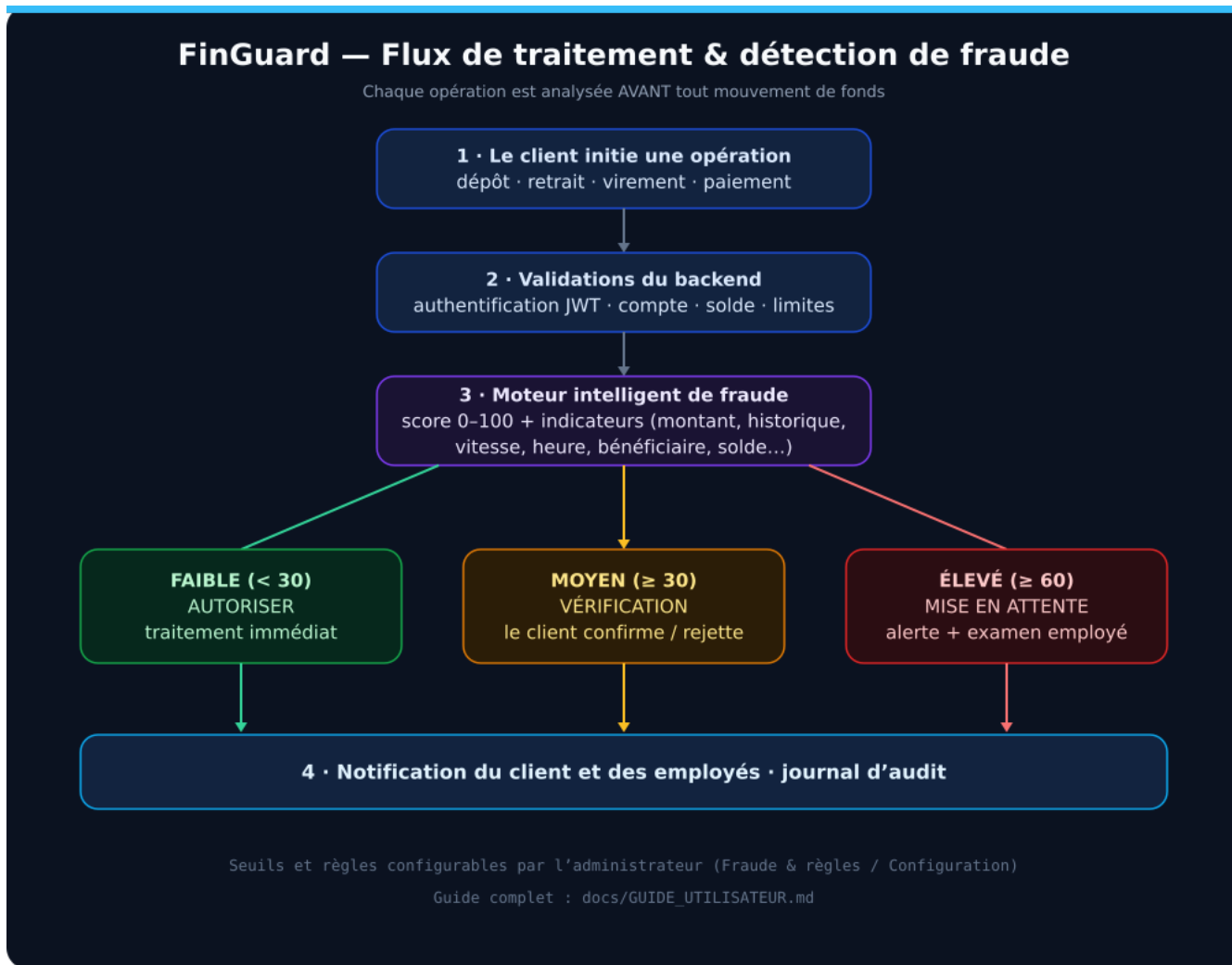
```
PORT=4000
JWT_SECRET=changez-moi-en-production
SQLITE_PATH=./data/finguard.db
```

4. Comptes de démonstration

Rôle	E-mail	Mot de passe
Administrateur	admin@finguard.com	Admin123!
Employé	marie.kouassi@finguard.com	Employe123!
Employé	emmanuel.njoya@finguard.com	Employe123!
Client	client@demo.com	Client123!
Client	amina.ndong@demo.com	Client123!

Le seed crée aussi 7 autres clients avec 30 jours d'historique, dont des cas prêts à examiner : un virement de 2 000 000 XAF **mis en attente** (score 85), un retrait **en attente de confirmation client**, un litige **en investigation**, un compte **gelé** et un cas **escaladé**.

5. Comprendre le flux de fraude



Chaque opération suit le circuit : initiation → validations → analyse de fraude →

décision → notification → audit. **Le moteur attribue un** score 0-100 à partir de règles configurables (montant exceptionnel, montant inhabituel vs historique, vitesse, heures inhabituelles, part du solde, nouveau bénéficiaire, volume quotidien) puis classe : **FAIBLE** → traitement immédiat ; **MOYEN (≥ 30)** → le client confirme ou rejette ; **ÉLEVÉ (≥ 60)** → mise en attente, alerte de fraude et examen employé.

6. Utilisation — Espace CLIENT

Connectez-vous avec `client@demo.com` / `Client123!`. Menu

latéral : Tableau de bord,

Transactions, Bénéficiaires, Litiges, Mon profil.

Tableau de bord

- **Solde total** et liste de vos comptes ; chaque compte possède un bouton **Relevé CSV** (historique complet, compatible Excel : séparateur ;, UTF-8).
- **Actions rapides** : Dépôt, Retrait, Virement, Paiement → ouvrent la modale Nouvelle transaction.
- **Mes dépenses (30 j)** : graphique en anneau de répartition de vos dépenses sortantes par type, avec montants et pourcentages.
- **Bannière de vérification** : si le moteur juge une de vos opérations inhabituelle (risque moyen), elle apparaît ici avec deux boutons : « **C'est moi, confirmer** » (l'opération est traitée) ou « **Je ne reconnais pas** » (un litige est créé).

Nouvelle transaction

1. Choisissez le type, le compte, le montant (et le compte bénéficiaire pour un virement — un **bénéficiaire enregistré** peut être sélectionné pour pré-remplir).
2. Cliquez sur « **Analyser et envoyer** » : le moteur de fraude analyse l'opération.
3. Résultat affiché immédiatement : OK — terminée, vérification requise ou mise en attente (examen employé), avec score et indicateurs.

Bénéficiaires

- **Ajouter** : nom + numéro de compte FG-XXXXXXX (+ banque optionnelle).
Un doublon est refusé (409).
- **Retirer** : icône corbeille, avec confirmation.

Transactions

- Filtres : recherche, type, statut, période.
- Clic sur une ligne → détail complet (score, indicateurs, statut).
- Bandeau « **Relevés de compte** » : téléchargement CSV par compte.

Litiges & profil

- Litiges : consultez vos litiges ; créez-en un via « Je ne reconnais pas » ou le formulaire (transaction + motif).
 - Mon profil : coordonnées, changement de mot de passe.
-

7. Utilisation — Espace EMPLOYÉ

Connectez-vous avec `marie.kouassi@finguard.com / Employe123!`. Menu : Tableau de

bord, Clients, Comptes, Transactions, Suspicieuses, Litiges, Rapports.

Tableau de bord + Simulateur de fraude

- KPIs (clients, comptes actifs, volume du jour, cas suspects) et courbe de volume.
- Bannière « **Simulateur de fraude** » → modale interactive : choisissez un compte, un type, un montant → le moteur rend un verdict (**score /100**, jauge, indicateurs, décision) **sans aucun mouvement de fonds**. Idéal pour former ou tester des seuils.

Clients & comptes

- Clients : liste, fiche détaillée (comptes, activité), création d'un client.
- Sur chaque compte : **Geler / Dégeler, Relevé CSV**, ajustement des limites (par opération / par jour).

Revue des transactions

- Transactions : toutes les opérations, détail avec **analyse de fraude** et **transactions connexes**.
- Pour une transaction **UNDER_REVIEW** : **Approuver** (traitement + crédit), **Rejeter** (motif requis) ou **Maintenir en attente**.
- Suspicieuses : file des cas à risque avec score ; Alertes : résoudre ou escalader à l'administration.

Litiges & rapports

- Litiges : changer le statut, ajouter des notes, résoudre (ou rejeter).
 - Rapports : activité quotidienne (volume, count, signalées).
-

8. Utilisation — Espace ADMIN

Connectez-vous avec `admin@finguard.com` / `Admin123!`.

Menu : Tableau de bord,

Employés, Clients, Fraude & règles, Configuration, Journaux d'audit, Rapports.

- **Tableau de bord** : vue système (clients, employés, volume, risques, croissance).
- **Employés** : créer un employé, activer/désactiver, **changer le rôle** (EMPLOYEE ↔ ADMIN).
- **Clients** : liste globale, fiche, activer/désactiver un client.
- **Fraude & règles** : statistiques de fraude, cas, et **configuration des règles** (activer/désactiver une règle, modifier ses points) ainsi que les **seuils** MOYEN/ÉLEVÉ.
- **Configuration** : paramètres système (limites globales, seuils...).
- **Journaux d'audit** : toutes les actions sensibles (connexions, approbations, gels, simulations, configurations) avec filtres.
- **Rapports** : graphiques globaux + carte « **Exports CSV** » : téléchargement de **toutes les transactions** (filtres statut/type) et du **journal d'audit**, au format Excel-compatible.

Sécurité (RBAC) : chaque espace est strictement réservé à son rôle. Un client qui force une URL `/employee/*` ou `/admin/*` reçoit une erreur **403**, un employé **403** sur `/admin/*`, et toute requête sans jeton **401**.

9. Scénarios à essayer (démonstration)

#	Manipulation	Résultat attendu
1	Client → Dépôt de 25 000 XAF	OK — Traitée immédiatement (risque faible)
2	Client → Virement de 2 500 000 XAF vers FG-10000102	Score élevé → UNDER_REVIEW ; l'employé l'approuve depuis Transactions
3	Client → Dépôt de 1 200 000 XAF	Risque moyen → bannière de confirmation client sur le tableau de bord
4	Client → Retrait de 99 999 999 XAF	KO — Refusé : « Limite par transaction dépassée »
5	Client → Bénéficiaires : ajouter deux fois le même compte	KO — Second ajout refusé (doublon)
6	Employé → Simulateur de fraude : 2 500 000 XAF	Verdict ÉLEVÉ, indicateurs détaillés, aucun mouvement de fonds
7	Admin → Fraude & règles : passer VERY_LARGE_AMOUNT à 30 pts puis relancer le scénario 2	Le score affiché change (les règles sont appliquées en direct)
8	Client → bouton sur un compte	Téléchargement du relevé CSV

10. Tests, documentation API et captures

Ressource	Emplacement
Documentation Swagger	<code>http://localhost:4000/api/docs</code>
Tests E2E (10 scénarios)	<code>backend/banking-api</code> → <code>npm run test:e2e</code>
Collection Postman (54 requêtes / 111 assertions) + environnement	<code>postman/</code> (voir <code>postman/README.md</code>)
Exécution CLI Newman + rapport HTML	<code>cd postman &&</code> <code>npm test</code> → <code>postman/reports/</code>
Captures d'écran des tests (synthèse + 5 dossiers + exécution complète)	<code>docs/tests-postman/</code>
Diagramme du flux de fraude	<code>docs/guide/flux-fraude.png</code>

11. Dépannage

Symptôme	Solution
node:sqlite inconnu / erreur au démarrage du backend	Installez Node 22+ (node -v)
Port 4000 already in use	ss -ltnp \ grep :4000 puis kill <pid> ; ou changez PORT dans .env
Site sans données / erreurs 500 sur /api	Vérifiez que l'API tourne (port 4000) et que npm run db:setup a été exécuté
État de démonstration « cassé » (soldes, comptes gelés...)	npm run db:reset côté backend puis redémarrez l'API
next build échoue après des changements	Arrêtez le serveur, supprimez .next, relancez npm run build && npm run start
Page blanche après mise à jour du code	Videz le cache navigateur ou relancez npm run dev
Erreurs d'installation npm	Supprimez node_modules et package-lock.json, puis npm install

Bonnes pratiques rappelées par l'application : mots de passe robustes fournis pour la démo uniquement ; en production, changez JWT_SECRET, utilisez HTTPS et un MySQL dédié (§3.2). Toute action sensible est journalisée dans l'audit.